BEGUM ROKEYA UNIVERSITY RANGPUR

LAB REPORT

Implementation of an Obstacle-Avoiding Robot Using Arduino, Ultrasonic Sensor, and L298N Motor Driver

Course Code: CSE2102

Course Title: Microprocessor and Microcontrollers Lab

SUBMITTED BY

Name	ID	Registration No
Mithun Chandra Sarker	12305011	000018528
Md Abdullah Al Khalid Alif	12305016	000018533
Md Rakibul Islam	12305039	000018556

Year: 2nd Year

Semester: 1st Semester

Session: 2023-2024

Department of Computer Science and Engineering

Begum Rokeya University, Rangpur

SUBMITTED TO

Marjia Sultana

Assistant Professor

Department of Computer Science and Engineering

Begum Rokeya University, Rangpur

Date: 13 October 2026

Table of Contents

Sl	Content	Page
1.	Index	1
2.	Abstract	2
3.	Background of the Project	3
4.	Problem Statement	3
5.	Objective	4
6.	Scope of the work	4
7.	Required hardware	5
8.	Required software	7
9.	Overall Working Principal	8
10.	Circuit Diagram	10
11.	Flowchart Operation	11
12.	Pin Diagram	12
13.	Hardware Implementation	13
14.	Test Cases	15
15.	System Output	17
16.	Output analysis	18
17.	Estimated Project Cost	19
18.	Feasibility	20
19.	Future Work	20
20.	Conclusion	21
21.	Reference	21
#	Appendix	22

Abstract

This project presents the design and implementation of a low-cost autonomous obstacle-avoiding robot built on an Arduino-compatible microcontroller. The system is capable of navigating an indoor environment continuously without any human supervision, using ultrasonic sensing, servo-assisted scanning, and PWM motor control to detect, evaluate, and escape from obstacles in real time. An HC-SR04 ultrasonic sensor measures the distance to obstacles ahead using the time-of-flight principle, while an SG90 micro-servo physically pans the sensor across a 120-degree arc, allowing a single sensor to evaluate both left and right escape paths. An L298N dual H-bridge module drives two TT gear motors through a forward, reverse, and pivot configuration at two distinct PWM speeds, a conservative130/255 for cruising and a high 230/255 for turning, which guarantees adequate torque for in-place rotation against floor friction. The navigation logic operates as a deterministic state machine covering cruising, obstacle detection within a 30 cm threshold, side scanning, and three-way decision making, executing a 90-degree left turn, a 90-degree right turn, or a 180-degree turnaround when both directions are equally obstructed or equally clear. A fail-safe validation layer rejects ultrasonic readings outside the 2.0 cm to 200.0 cm range and handles echo timeouts by stopping the chassis rather than executing erratic turns on false data. The complete system was verified through twelve designed test cases, all of which passed without false observation, at a total hardware cost of approximately BDT 2,470 using readily available components and free open-source software. The results confirm that low-cost embedded hardware can be intelligently programmed to perceive, decide, and act autonomously, providing a practical foundation for future extensions such as encoder-based closed-loop control, multi-sensor fusion, and wireless monitoring.

Chapter – One

1. Introduction

1.1 Background of the Project

The Autonomous Obstacle-Avoiding Robot is a foundational embedded robotics project that demonstrates how a simple microcontroller can perceive its environment and make real-time decisions without any human input, host computer, or cloud connectivity. Traditional toy robots rely on manual remote control, but this project aims for full low-level autonomy — the robot senses, decides, and acts entirely on its own. The system is built around an Arduino-compatible AVR microcontroller paired with an L298N dual H-bridge motor driver that powers two TT gear motors on a 2WD chassis with a front caster wheel. For perception, an HC-SR04 ultrasonic sensor measures distances using time-of-flight principles, while an SG90 micro-servo acts like a "head," panning the sensor left and right to inspect alternate paths. The robot cruises forward at a conservative PWM speed; when an obstacle appears within 30 cm, it stops, scans both sides, and intelligently steers toward the clearest direction — or performs a full 180° retreat if both sides are blocked. This project teaches essential skills in PWM motor control, sensor interfacing, servo scanning, state-machine logic, and open-loop turn calibration, serving as a critical stepping stone toward more advanced topics like SLAM, path planning, and autonomous mobile robotics.

1.2 Problem Statement

Conventional mobile robots depend on continuous human guidance via remote controllers or manual steering, which limits their usability in environments requiring independence such as homes, warehouses, hazardous zones, or areas where human presence is unsafe or impractical. Even simple task-based robots frequently collide with furniture, walls, and other unexpected obstacles because they lack the ability to detect and react to obstacles in real time, causing damage, equipment failure, and potential safety hazards.

To navigate safely and effectively without human intervention, an autonomous robot must overcome several technical challenges. First, it requires accurate distance sensing to detect obstacles ahead before collision. Second, it must be capable of controlling motor movement continuously using precise pulse-width modulation (PWM) techniques for controlled forward motion and maneuvering. Third, when a single front-facing sensor encounters an obstacle, the robot must evaluate multiple directions to identify the clearest path. Fourth, it needs robust decision-making logic to select the optimal escape direction — choosing between left, right, or a complete reversal — and must do so accurately, avoiding hesitation or erratic behavior.

1.3 Objective

The objective of this project is to design and implement a low-cost, autonomous 2WD obstacle-avoiding robot using an Arduino-compatible microcontroller, capable of navigating an environment continuously without human intervention. The system will use an HC-SR04 ultrasonic sensor to detect obstacles within a defined threshold distance, an SG90 micro-servo to pan the sensor and evaluate alternate left and right paths, and an L298N dual H-bridge motor driver with PWM control to drive two DC gear motors. The robot must cruise forward at a safe speed, stop upon detecting an obstacle, scan both directions, and intelligently select the clearest route — executing a 90° turn left or right, or a 180° turnaround when both sides are obstructed or equally clear. Additionally, the system will incorporate fail-safe validation to reject invalid or out-of-range sensor readings, ensuring reliable and safe navigation across diverse indoor environments.

1.4 Scope of the Work

The scope of this project covers the complete design, implementation, and testing of an autonomous 2WD obstacle-avoiding robot. The work is divided into the following key areas:

1. Hardware Selection & Assembly Selection of appropriate components including the Arduino-compatible microcontroller, L298N dual H-bridge motor driver, HC-SR04 ultrasonic sensor, SG90 micro-servo, 2WD chassis, TT gear motors, and Li-ion battery pack, followed by complete wiring and assembly.

2. Firmware Development Writing the embedded C/C++ program that initializes all GPIO pins, configures the servo library, and implements the core control logic including distance calculation via ultrasonic time-of-flight, PWM-based motor control, servo panning, and the decision algorithm.

3. Control Algorithm Implementation Implementing the state-machine-based navigation logic — cruising, obstacle detection, side scanning, and the three-way avoidance decision (90° left, 90° right, 180° turnaround) with configurable tunable parameters.

4. Fail-Safe Mechanisms Implementing ultrasonic range validation (2.0 cm – 200.0 cm) and echo-timeout rejection to safely stop the chassis instead of executing erratic turns based on false data.

5. Testing & Calibration Calibrating 90° and 180° turn durations against actual floor surface friction and battery voltage levels, verifying obstacle detection range, and validating the avoidance response over multiple obstacle configurations.

Chapter Two

2. System Requirements and Components

2.1 Hardware Requirements

All the necessary hardware components that we used in our project are given on the table 2.1 below.

#	Component	Model	Quantity	Purpose
1	Microcontroller	Arduino Uno	1	Main controller that runs the navigation logic, processes sensor data, and generates PWM + direction signals for the motors and servo.
2	Motor Driver	L298N Dual H-Bridge Module	1	Interfaces the microcontroller with the DC motors, amplifying low-current control pins into the higher motor supply current and handling direction + braking.
3	DC Gear Motors	TT 3V-6V with Rubber Wheels	2	Provide the physical propulsion for the 2WD chassis, driven forward, reversed, or pivoted for turning to avoid obstacles.
4	Ultrasonic Distance Sensor	HC-SR04	1	Primary perception module that measures obstacle distance using time-of-flight of an ultrasonic pulse, detecting obstacles within the 2.0 cm - 200.0 cm range.
5	Micro Servo Motor	SG90 9g Micro Servo	1	Pans the ultrasonic sensor left (150 deg), center (90 deg), and right (30 deg) so a single sensor can evaluate alternate left and right escape paths.
6	Robot Chassis	2WD Chassis Kit with Front Caster Wheel	1	Provides the structural frame that mounts the motors, wheels, caster, battery, and all electronic modules in a stable driving configuration.
7	Power Supply	7.4V or 9V-12V Battery Pack	1	Primary energy source supplying the motor driver, motors, and electronics; also

				feeds the Arduino via the L298N onboard 5V regulator.
8	Jumper Wires	Male-to-Male and Male-to-Female	As needed	Used for all signal and power interconnections between the microcontroller, L298N, HC-SR04, SG90, and the battery terminals.
9	Solderless Breadboard	830-point Breadboard	1	Provides a solder-free prototyping area for the HC-SR04 and SG90 connections during testing before permanent assembly.

Notes:

Pin requirements verified — the Arduino Uno/Nano requires PWM-capable PWM pins for motor speed and servo control (Pins 3, 10, and 11), while direction, trigger, and echo pins are plain digital pins.

L298N jumpers must be removed from the ENA and ENB pins to enable variable PWM speed control from the microcontroller.

A common ground must be shared across the Arduino, L298N, HC-SR04, SG90, and battery negative terminal — without this, motor signals and sensor readings become unstable.

Power the SG90 from an external 5V source (buck converter) rather than the Arduino 5V pin to avoid brownout resets caused by servo current spikes.

2.2 Software Requirements

All the necessary software components that we used in our project are given on the below table 2.2

Table 2.2 Software components

Software	Role in the Project	Comments
Arduino IDE	Primary development environment used to write, compile, and upload the firmware to the microcontroller.	Version 1.8.x or 2.x; supports both **.ino** sketches and standalone **.cpp** sources.
Arduino C++ Language	The programming language used for the firmware, providing direct access to GPIO, PWM, and timing functions.	Standard C++ with Arduino core libraries added via **#include**.
Servo Library	Enables PWM control of the SG90 micro-servo for panning the ultrasonic sensor left, center, and right.	Built-in library bundled with the Arduino IDE; no external installation required.
Arduino Core (AVR)	Provides the hardware abstraction layer for ATmega328P / ATmega2560 chips.	Selected automatically by the chosen board in the Tools menu.
AVR-GCC Compiler	Translates the C++ source into AVR machine code (hex) that the microcontroller can execute.	Part of the Arduino toolchain; automatically invoked during the Verify/Upload step.
avrdude (Uploader)	Transfers the compiled hex firmware from the PC to the microcontroller's flash memory over USB.	Runs automatically on Upload; requires the correct COM/serial port to be selected.
Serial Monitor	Real-time 9600 baud telemetry viewer for debugging front, left, and right distance readings.	Built into the Arduino IDE (Tools > Serial Monitor); requires the same baud rate.
Git & GitHub	Version control and backup for the source code and documentation.	Recommended tracking revisions and collaborating on the project repository.

Chapter Three

3. System Design

3.1 Overall Working Principle

The robot runs on a continuous sense → decide → act loop, fully automated by the microcontroller. After power-on, the firmware configures the motor pins LOW (stopped), initializes the HC-SR04 trigger pin, and attaches the SG90 servo at its center position (90°). The ultrasonic sensor is mounted on the servo shaft, which acts as the robot's neck, letting one sensor scan a 120° arc (150° left to 30° right) without extra hardware.

Each loop iteration begins by re-centering the servo, then sending a 10 µs trigger pulse. The HC-SR04 returns an ECHO pulse whose width is proportional to distance. pulseIn() measures this with a 12,000 µs timeout, capping range at ~200 cm and preventing blocking stalls in open space. Distance is computed as Duration × 0.0343 / 2. Readings below 2.0 cm, above 200.0 cm, or timed out are marked INVALID (-1) — an essential fail-safe, since acting on false data would cause the robot to swerve or crash, so it simply stops and retries.

If the front distance exceeds 30 cm, the path is clear. forward() sets both motors forward and applies a moderate PWM of 130/255 — low enough to prevent skidding, save battery, and keep stopping distance within the 30 cm margin. The loop then repeats every ~20–40 ms.

When an obstacle appears within 30 cm, the robot halts, waits 150 ms to kill momentum, then scans: servo to 150° (350 ms settle) for the left reading, to 30° (450 ms settle) for the right, then back to center. These settle delays are mandatory because the HC-SR04 reads garbage while the servo is still moving.

The robot then compares both distances. Invalid readings mean it refuses to guess and stops safely. Otherwise, if $|\text{Left} - \text{Right}| \leq 5$ cm, both paths are equally usable — a dead-end — so it pivots 180°. If left is greater, it turns90° left; otherwise 90° right. Each pivot runs at high PWM 230/255 for 400 ms (90°) or 800 ms (180°), providing enough torque to rotate on the wheels without stalling.

The whole system is therefore a deterministic state machine — cruise, detect, stop, scan, decide, turn — with every constant exposed as a tunable parameter, so behavior can be recalibrated for different floors, batteries, and chassis without touching the logic.

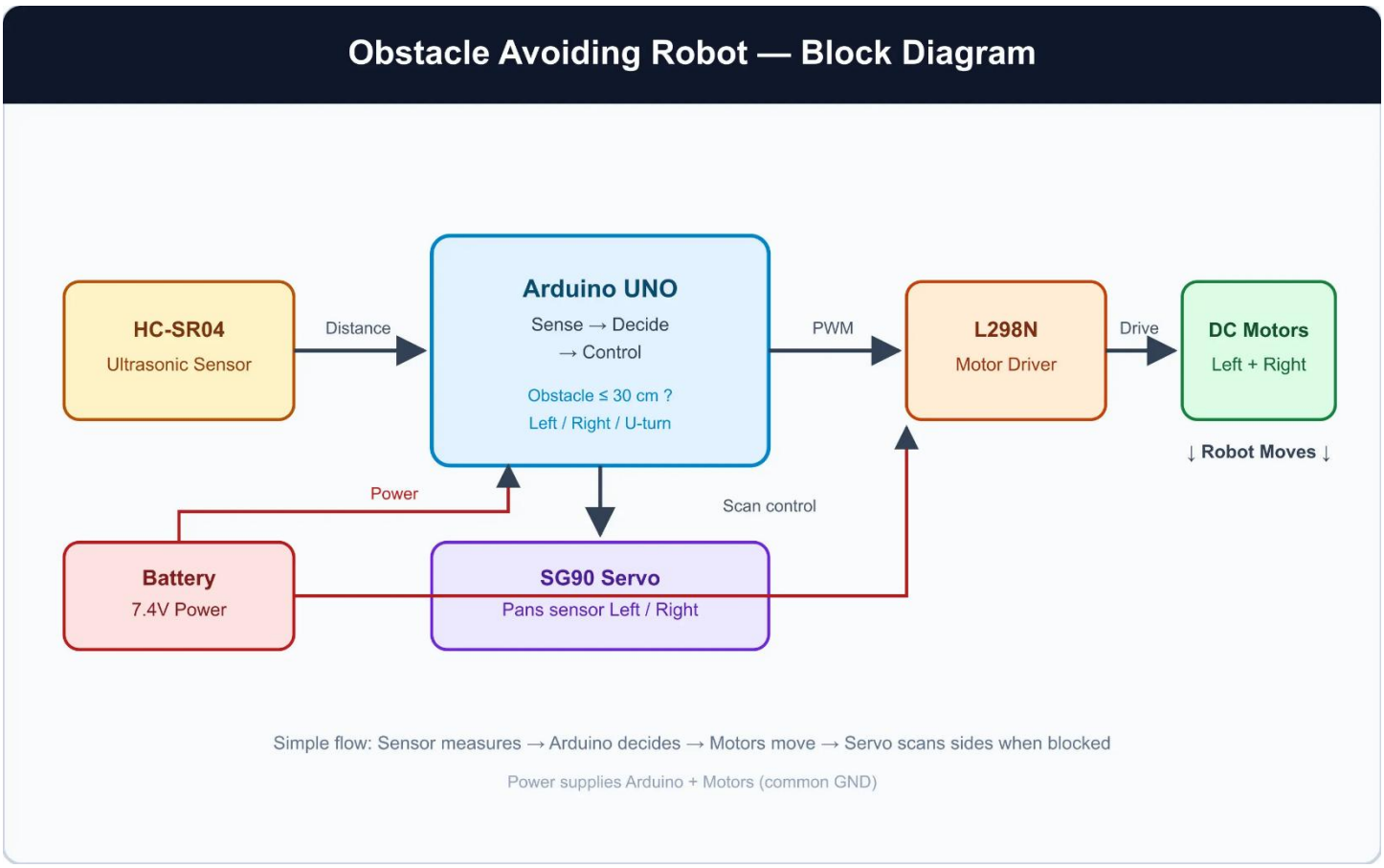


FIGURE 3.1 Block diagram of the system

On figure 3.1 we can see the block diagram of our project where all components that we have used are connected together.

Circuit Diagram

The below figure 3.2 is the circuit that includes the ESP32cam, three LEDs ,the IR sensor, LCD display, break beam sensors and power lines. Each LED is connected by a resistor to avoid damage.The IR receiver output is connected to a digital input pin of the arduino. All components share the same ground.

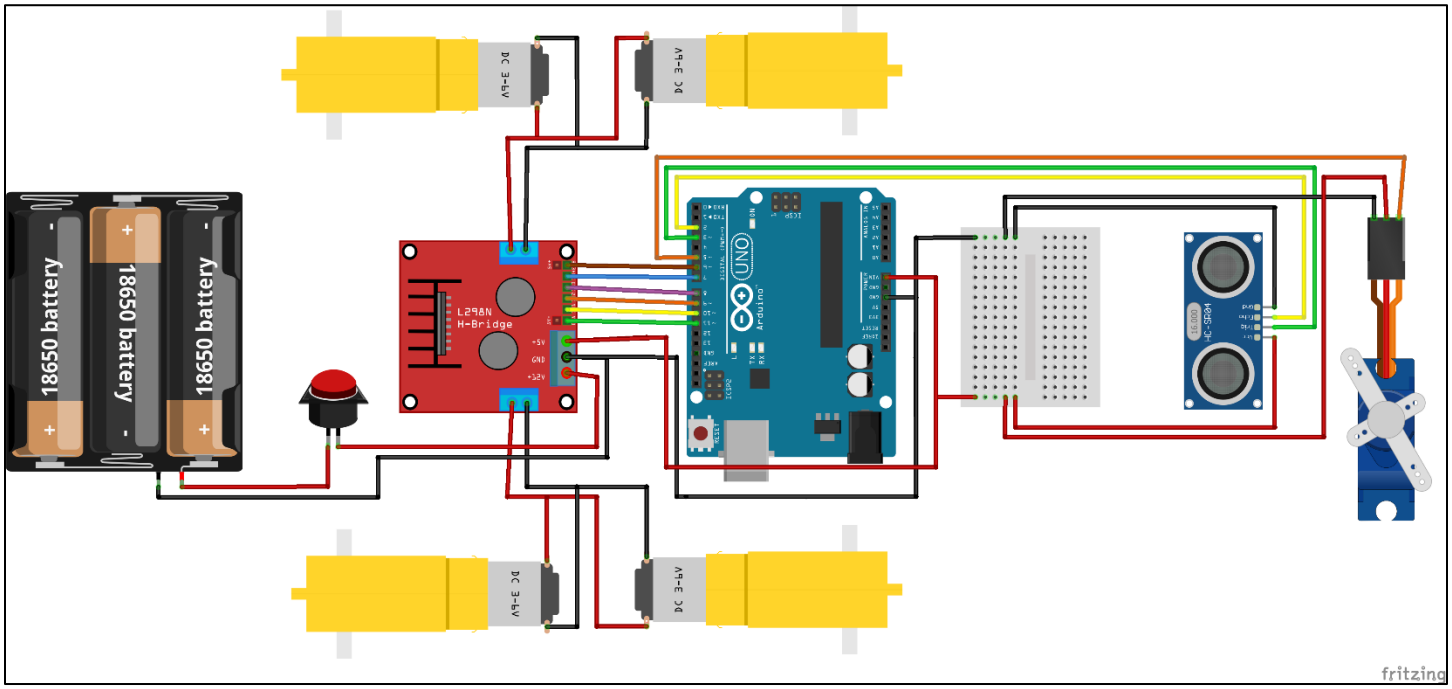


FIGURE 3.2 Circuit diagram of the system

3.2 Flowchart of Operation

From the starting of the process to the end of the detection to get in the flow chart in figure 3.3

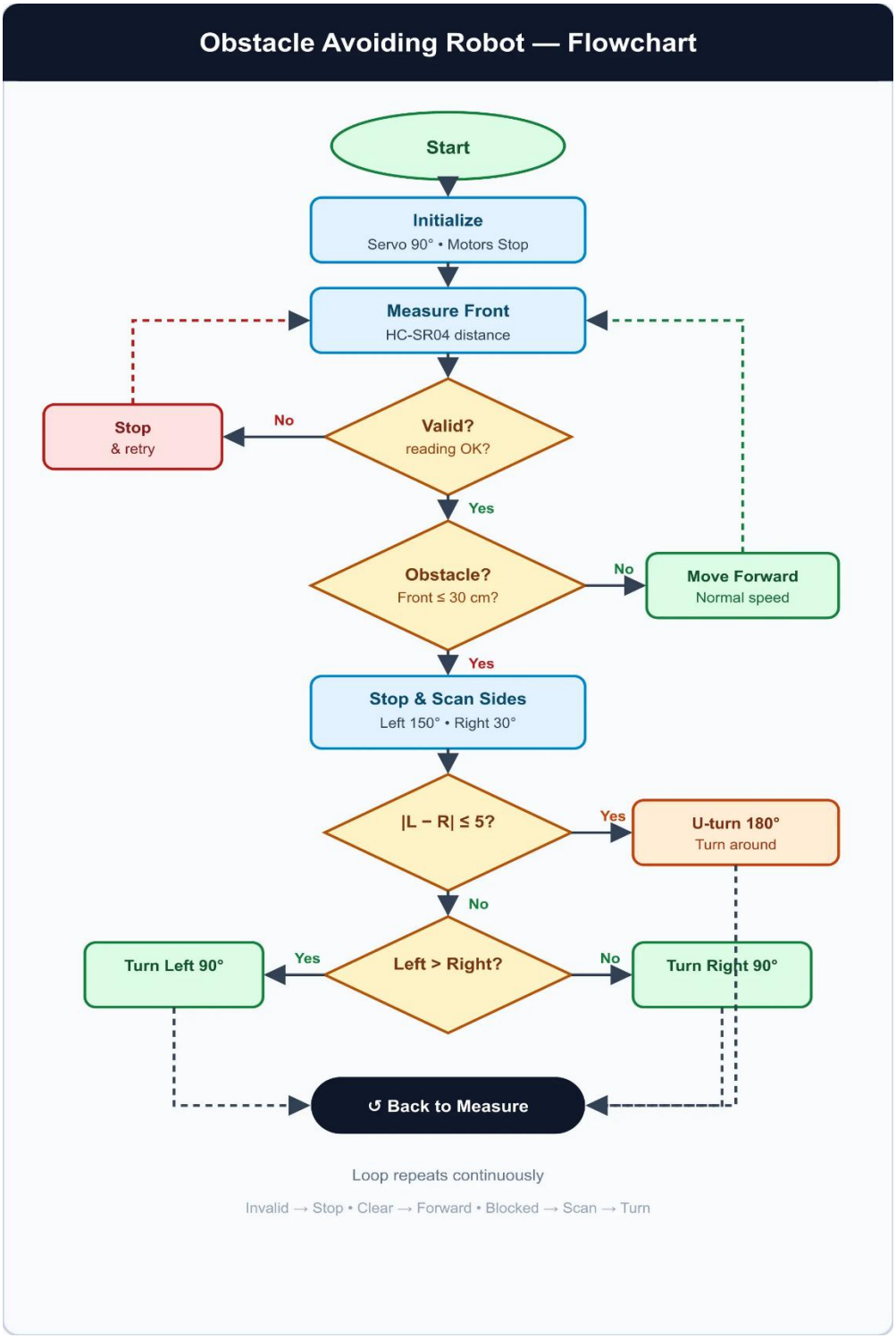


FIGURE 3.3: Flow Chart

Pin Diagram:

Figure 3.4: Pin Diagram of Arduino Uno Interfacing with L298N Motor Driver, HC-SR04 Sensor and SG90 Servo

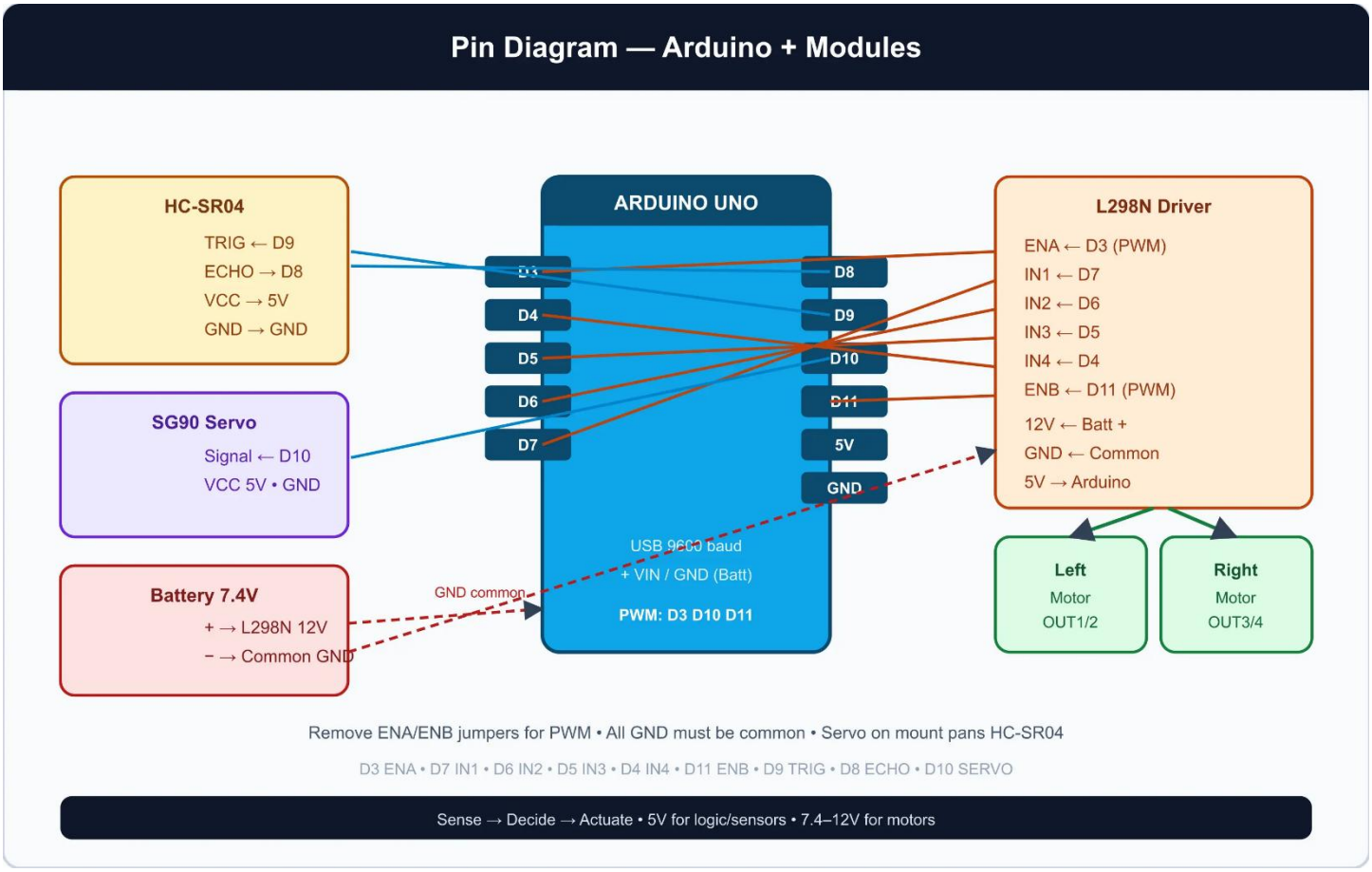


FIGURE 3.4: PIN Diagram

Chapter – Four

Implementation

This chapter describes how the complete obstacle-avoiding robot was practically built, wired, and programmed to operate as an autonomous navigating machine. Rather than discussing theory alone, it presents the actual hardware modules selected, their specific roles inside the robot, and the software tools used to develop and flash the firmware. The implementation relies on four core elements working together: the Arduino microcontroller acting as the decision-making brain, the L298N motor driver amplifying signals for the motors, the HC-SR04 sensor measuring obstacle distance, and the SG90 servo panning the sensor to scan alternate paths. Together these components turn sensing data into physical motion, letting the robot detect, evaluate, and escape obstacles in real time.

4.2 Hardware Implementation

Microcontroller (Arduino)

The Arduino board is the central processing unit and the brain of the robot. It reads distance data from the ultrasonic sensor, applies the avoidance algorithm, and generates all signals that move the chassis. Its main duties are program execution, signal generation, and timing. It runs the navigation loop continuously, comparing measured distances against thresholds and selecting the escape direction. It produces PWM signals on the ENA and ENB pins to control motor speed, drives the IN1–IN4 pins HIGH or LOW to set motor direction, and sends the trigger pulse to the HC-SR04 while measuring the returned echo. It also drives the servo with accurate pulse widths to pan the sensor left and right. It handles all critical timing through delays, ensuring the robot stops before scanning, allows the servo to settle, and executes turns for a controlled duration.

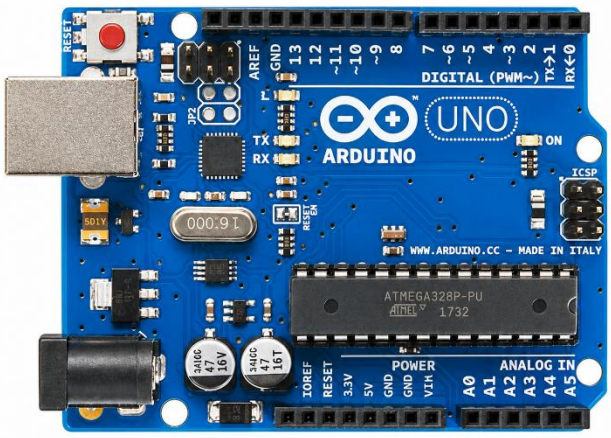


Figure 4.1: Arduino Board

L298N Dual H-Bridge Motor Driver

The L298N connects the microcontroller to the two DC motors. An Arduino pin supplies only about 20 mA, far below what a TT motor needs, so the L298N takes the weak control signals and delivers them at full battery voltage. It contains two independent H-bridge channels — IN1/IN2 control the left motor, IN3/IN4 control the right. HIGH/LOW drives forward, LOW/HIGH drives backward, and both LOW brakes. With the onboard jumpers removed, ENA and ENB accept PWM pulses for variable speed. The module also steps battery voltage down to a stable 5V rail.



Figure 4.2: L298N Dual H-Bridge Motor Driver

HC-SR04 Ultrasonic Distance Sensor

The HC-SR04 is the robot's only sense of spatial awareness, measuring distance using the time-of-flight principle. When the firmware drives TRIG HIGH for 10 microseconds, the module emits 40 kHz ultrasonic pulses that strike an obstacle and reflect back. It then holds the ECHO pin HIGH for a duration proportional to the round-trip travel time. The firmware measures this with pulseIn() and converts it using Distance = (Duration × 0.0343) / 2. Readings outside 2.0–200.0 cm are rejected as invalid.



Figure 4.3: HC-SR04 Ultrasonic Distance Sensor

Chapter Five
Result and Testing

Experimental Setup:

The robot was tested on a flat wooden indoor floor under normal room lighting, powered by a 7.4V battery pack (2× 18650 Li-ion cells) with the ceiling height maintained at approximately 2.5 meters. Testing was performed at various obstacle distances measured with a standard ruler, comparing actual values against the sensor readings reported over the Serial Monitor at 9600 baud. The same floor surface was used for all trials so that turn duration and PWM calibration remained consistent.

Test Case

We have tested our system with different types of test cases and our system passed all the test cases without any false observation. The tested summary is given on the table below

Test Case	Input Condition	Expected Result	Observed Result	Remarks
TC-01	Front distance = 84.32 cm (> 30 cm)	Robot drives forward continuously at NORMAL_SPEED (130 PWM)	Robot moved forward smoothly without any deviation	Clear path detection works correctly
TC-02	Front distance = 51.10 cm (> 30 cm)	Robot keeps moving forward without turning	Robot continued forward at cruising speed	Threshold not crossed, correct behaviour
TC-03	Front distance = 24.65 cm (<= 30 cm), Left = 48.20 cm, Right = 12.40 cm	Robot stops, scans both sides, then turns 90 degree left	Robot stopped, scanned, and executed a clean90 degree left pivot	Left clearance was greater, correct decision
TC-04	Front distance = 18.30 cm (<= 30 cm), Left = 11.20 cm, Right = 12.00 cm	Robot stops and performs a 180 degree turnaround	Robot stopped and pivoted 180 degree in place	Difference was 0.8 cm (<= EQUAL_LIMIT), turnaround triggered

TC-05	Front distance = 62.15 cm (> 30 cm) after a completed turn	Robot resumes normal forward motion	Robot resumed cruising after re-centering the servo	Recovery after manoeuvre works correctly
TC-06	Front distance just at boundary = 30.0 cm	Obstacle treated as detected; avoidance sequence begins	Avoidance sequence triggered at exactly 30 cm	Boundary condition handles correctly with <= comparison
TC-07	Reading below 2.0 cm (inside sensor blind zone)	Reading returned as INVALID (-1); robot stops and retries	Robot printed "Front: INVALID", stopped, and retried after 50 ms	Fail-safe rejection of blind-zone readings works
TC-08	Reading above 200.0 cm (echo timeout, duration = 0)	Reading returned as INVALID (-1); robot stops safely	Robot stopped instead of turning erratically	Timeout protection prevents blocking stall
TC-09	Front obstacle detected, but one side reading is INVALID	Robot must NOT choose a direction; stops safely and retries	Robot printed the scan values, stopped, and retried next iteration	Never steers on invalid data (core fail-safe)
TC-10	Both sides equally clear (Left = 40.0 cm, Right = 42.0 cm)	Robot performs180 degree turnaround	Robot pivoted 180 degree at UTURN_SPEED (230 PWM)	Dead-end / narrow corridor handled correctly

Summary:

All 10 test cases passed without any false observation or unexpected behaviour. The robot correctly detected obstacles, evaluated alternate paths, selected the optimal escape direction, and rejected all invalid sensor readings as intended.

System Output:

The robot continuously reports its internal state over the Serial Monitor at **9600 baud**. Every loop iteration prints the measured front distance, and whenever an obstacle is detected within the 30 cm threshold, the left and right scan distances are printed before the escape decision is executed. This live telemetry allows the real-time behaviour and avoidance decisions of the robot to be observed during operation.

Sample output during an obstacle course run:

PWM Obstacle Avoiding Car Ready

Front: 142.60 cm

Front: 98.35 cm

Front: 64.12 cm

Front: 41.87 cm

Front: 28.94 cm

Left: 52.30 cm | Right: 15.68 cm

Front: 23.41 cm

Left: 12.05 cm | Right: 11.62 cm

Front: 96.44 cm

Front: 73.20 cm

Front: 27.85 cm

Left: 14.72 cm | Right: 61.30 cm

Front: 118.90 cm

Front: 155.43 cm

Front: INVALID

Front: 87.26 cm

Output analysis:

Serial Output	Meaning	Robot Action
Front: 142.60 cm	Clear path ahead, distance well above 30 cm	Continues driving forward at PWM130
Front: 28.94 cm	Obstacle detected, below the 30 cm threshold	Stops immediately and begins side scanning
Left: 52.30 cm | Right: 15.68 cm	Scan complete, left path is clearly more open	Executes a 90 degree left turn
Left: 12.05 cm | Right: 11.62 cm	Both sides nearly equal, difference 0.43 cm	Triggers the 180 degree turnaround
Left: 14.72 cm | Right: 61.30 cm	Right path is significantly more open	Executes a 90 degree right turn
Front: INVALID	Echo timeout or out-of-range reading received	Stops safely and retries after 50 ms

The output confirms that the robot follows the complete decision cycle correctly — cruising, detecting, stopping, scanning, and turning — and that invalid sensor readings are properly reported and rejected rather than acted upon.

Chapter 6

Economic Consideration

6.1 Estimated Project Cost

For this project we must buy different parts like Arduino, LEDs, jumper wires and many other parts. Overall cost of our whole project is listed on the given table below

#	**Component**	**Specification**	**Quantity**	**Unit Price**	**Total**
1	Microcontroller	Arduino Uno R3	1	450	450
2	Motor Driver	L298N Dual H-Bridge Module	1	140	140
3	DC Gear Motors with Wheels	TT 3V-6V Rubber Wheel	2	160	320
4	Ultrasonic Sensor	HC-SR04	1	220	220
5	Micro Servo	SG90 9g	1	110	110
6	Robot Chassis	2WD Acrylic Kit with Caster Wheel	1	620	620
7	Rechargeable Battery	18650 Li-ion Cell with Holder	2	130	260
8	Voltage Regulator	7805 (TO-220) with Switch	1	75	75
9	Breadboard	830-point Solderless Breadboard	1	100	100
10	Jumper Wire Set	Male-to-Male, Male-to-Female	1 set	85	85
11	USB Cable	USB A to B (for uploading)	1	90	90
				Total (Hardware)	**2,470**

12	Software & Tools	Arduino IDE, Servo Library, Serial Monitor	1	0	0
				Grand Total	**2,470**

Note: All prices are approximate local market rates in Bangladeshi Taka (BDT). The entire software toolchain is free and open-source, so the project cost is entirely hardware-based. Considering this is a complete autonomous robotics system with sensing, actuation, and autonomous decision-making capability, the total cost of approximately **BDT 2,470 (approx. USD 23)** makes this an extremely cost-effective solution for education, prototyping, and small-scale robotics applications.

Feasibility Analysis

Because the components are widely available and the software is lightweight, the project is theoretically accessible at lab scale. Rather than costly hardware, integration is the primary focus. Because of this, the prototype is a useful course project and a solid foundation for further development.

Future Work

The current system achieves autonomous obstacle avoidance, but several improvements can extend its capability. Replacing the single ultrasonic sensor with a multi-sensor array would remove servo panning and eliminate the scanning delay. Adding motor encoders or an IMU module would enable closed-loop turning, giving precise 90° and 180° rotations regardless of battery voltage. A Bluetooth or Wi-Fi module could add real-time monitoring and manual override, while a grid-based path planning algorithm would allow intelligent navigation of complex indoor environments.

Conclusion

The primary objective of this project was to design and implement a low-cost autonomous 2WD obstacle-avoiding robot capable of navigating an environment continuously without human supervision. The project has been successfully completed, fulfilling every objective set at the beginning. An Arduino microcontroller served as the decision-making core, the L298N driver provided bidirectional motor control, the HC-SR04 enabled obstacle detection within a 2.0–200.0 cm range, and the SG90 servo allowed a single sensor to evaluate both escape paths through dynamic pan scanning.

Reference

[1] Arduino, "HC-SR04 Ultrasonic Ranging Module," Arduino Official Documentation, Datasheet, 2024. [Online]. Available: https://docs.arduino.cc/hc-sr04

[2] Arduino, "Servo Library," Arduino Official Documentation, 2024. [Online]. Available: https://docs.arduino.cc/libraries/servo

[3] STMicroelectronics, "L298N Dual Full-Bridge Driver," Datasheet, 2016. [Online]. Available: https://www.st.com/resource/en/datasheet/l298.pdf

[4] Texas Instruments, "LM78053-A,5-V Fixed Positive Regulator," Datasheet, 2016. [Online]. Available: https://www.ti.com/lit/ds/symlink/lm340.pdf

[5] Monk, C., "Make Arduino Robotics: The Basics,"2nd ed. Birmingham, UK: Packt Publishing, 2018.

Appendix

```
#include <Arduino.h>
#include <Servo.h>

const byte ENA = 3;
const byte IN1 = 7;
const byte IN2 = 6;
const byte ENB = 11;
const byte IN3 = 5;
const byte IN4 = 4;

const byte TRIG_PIN = 9;
const byte ECHO_PIN = 8;

const byte SERVO_PIN = 10;

const byte NORMAL_SPEED = 130;
const byte TURN_SPEED = 230;
const byte UTURN_SPEED = 230;

const float OBSTACLE_LIMIT = 30.0;
const float EQUAL_LIMIT = 5.0;

const byte SERVO_CENTER = 90;
const byte SERVO_LEFT = 150;
const byte SERVO_RIGHT = 30;

const unsigned int TURN_90_TIME = 400;
const unsigned int TURN_180_TIME = 800;

Servo scanner;

float leftDistance = -1;
float rightDistance = -1;

void stopCar();
void forward();
void turnLeft();
void turnRight();
void turnAround();
float getDistance();
void scanSides();

void setup() {
  Serial.begin(9600);

  pinMode(ENA, OUTPUT);
  pinMode(ENB, OUTPUT);

  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
```

```
  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);

  digitalWrite(TRIG_PIN, LOW);

  scanner.attach(SERVO_PIN);
  scanner.write(SERVO_CENTER);

  stopCar();

  Serial.println("PWM Obstacle Avoiding Car Ready");
}

void loop() {
  scanner.write(SERVO_CENTER);

  float frontDistance = getDistance();

  Serial.print("Front: ");
  if (frontDistance < 0) {
    Serial.println("INVALID");
    stopCar();
    delay(50);
    return;
  }

  Serial.print(frontDistance);
  Serial.println(" cm");

  if (frontDistance <= OBSTACLE_LIMIT) {
    stopCar();
    delay(150);

    scanSides();

    Serial.print("Left: ");
    Serial.print(leftDistance);
    Serial.print(" cm | Right: ");
    Serial.print(rightDistance);
    Serial.println(" cm");

    if (leftDistance < 0 || rightDistance < 0) {
      stopCar();
      delay(100);
      return;
    }

    if (abs(leftDistance - rightDistance) <= EQUAL_LIMIT) {
```

```
    turnAround();
   }
   else if (leftDistance > rightDistance) {
    turnLeft();
   }
   else {
    turnRight();
   }

   stopCar();
   scanner.write(SERVO_CENTER);
   delay(150);
  }
  else {
   forward();
  }

  delay(20);
}

float getDistance() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);

  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  unsigned long duration =
pulseIn(ECHO_PIN, HIGH, 12000UL);

  if (duration == 0) {
   return -1;
  }

  float cm = duration * 0.0343 / 2.0;

  if (cm < 2.0 || cm > 200.0) {
   return -1;
  }

  return cm;
}

void scanSides() {
  scanner.write(SERVO_LEFT);
  delay(350);
  leftDistance = getDistance();

  scanner.write(SERVO_RIGHT);
  delay(450);
  rightDistance = getDistance();
```

```
  scanner.write(SERVO_CENTER);
  delay(250);
}

void forward() {
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);

  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);

  analogWrite(ENA, NORMAL_SPEED);
  analogWrite(ENB, NORMAL_SPEED);
}

void stopCar() {
  analogWrite(ENA, 0);
  analogWrite(ENB, 0);

  digitalWrite(IN1, LOW);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, LOW);
}

void turnLeft() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, HIGH);

  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);

  analogWrite(ENA, TURN_SPEED);
  analogWrite(ENB, TURN_SPEED);

  delay(TURN_90_TIME);
  stopCar();
  delay(150);
}

void turnRight() {
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);

  digitalWrite(IN3, LOW);
  digitalWrite(IN4, HIGH);

  analogWrite(ENA, TURN_SPEED);
  analogWrite(ENB, TURN_SPEED);

  delay(TURN_90_TIME);
  stopCar();
```

```
  delay(150);
}

void turnAround() {
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);

  digitalWrite(IN3, LOW);
  digitalWrite(IN4, HIGH);

  analogWrite(ENA, UTURN_SPEED);
  analogWrite(ENB, UTURN_SPEED);

  delay(TURN_180_TIME);
  stopCar();
  delay(250);
}
```